```
import torch

inputs = torch.tensor(
    [
        [0.43, 0.15, 0.89],  # Your     (x^1)
        [0.55, 0.87, 0.66],  # journey  (x^2)
        [0.57, 0.85, 0.64],  # starts   (x^3)
        [0.22, 0.58, 0.33],  # with     (x^4)
        [0.77, 0.25, 0.10],  # one      (x^5)
        [0.05, 0.80, 0.55],
    ]  # step     (x^6)
)

print("Input Shape:", inputs.shape)
```

```
Input Shape: torch.Size([6, 3])
```

```
# Input size = 3
d_in = inputs.shape[1]
# Output size of Wq , Wk, Wv weight matrices
d_out = 2

torch.manual_seed(123)

W_query = torch.nn.Parameter(torch.rand(d_in, d_out),
requires_grad=False)
W_key = torch.nn.Parameter(torch.rand(d_in, d_out),
requires_grad=False)
W_value = torch.nn.Parameter(torch.rand(d_in, d_out),
requires_grad=False)

print("W_query - 3x2 matrix :", W_query)
print("W_key - 3x2 matrix : ", W_key)
print("W_value -  3x2 matrix : ", W_value)
```

```
W_query - 3x2 matrix : Parameter containing:
tensor([[0.2961, 0.5166],
        [0.2517, 0.6886],
        [0.0740, 0.8665]])
W_key - 3x2 matrix :  Parameter containing:
tensor([[0.1366, 0.1025],
        [0.1841, 0.7264],
        [0.3153, 0.6871]])
W_value -  3x2 matrix :  Parameter containing:
tensor([[0.0756, 0.1966],
        [0.3164, 0.4017],
        [0.1186, 0.8274]])
```

```
keys = inputs @ W_key
values = inputs @ W_value
queries = inputs @ W_query
```

```
print("keys.shape:", keys.shape)
print("values.shape:", values.shape)
print("queries.shape:", queries.shape)
```

```
keys.shape: torch.Size([6, 2])
values.shape: torch.Size([6, 2])
queries.shape: torch.Size([6, 2])
```

```
# Computing the attention score for the second word in the input
sequence (journey)

query_2 = queries[1]  # query for the second word (journey)
keys_2 = keys[1]  # keys for the second word (journey)

attn_Score_22 = query_2.dot(keys_2)  # dot product of query and key
for the second word
print("Attention Score for the second word (journey):",
attn_Score_22.item())
```

```
Attention Score for the second word (journey): 1.8523844480514526
```

```
# Again, we can generalize this computation to all attention scores
(for all the key vectors with respect to query_2 vector) via matrix
multiplication:

attn_scores_2 = query_2 @ keys.T

# All attention scores for given query
# (1x2) * (2*6) = (1*6) attention score (attention score of the second
input(query vector in this case) with all other words in the inputs
(key vectors))

print(attn_scores_2)
```

```
tensor([1.2705, 1.8524, 1.8111, 1.0795, 0.5577, 1.5440])
```

```
all_attn_scores = queries @ keys.T  # omega
print(all_attn_scores)
```

```
tensor([[0.9231, 1.3545, 1.3241, 0.7910, 0.4032, 1.1330],
        [1.2705, 1.8524, 1.8111, 1.0795, 0.5577, 1.5440],
        [1.2544, 1.8284, 1.7877, 1.0654, 0.5508, 1.5238],
        [0.6973, 1.0167, 0.9941, 0.5925, 0.3061, 0.8475],
        [0.6114, 0.8819, 0.8626, 0.5121, 0.2707, 0.7307],
        [0.8995, 1.3165, 1.2871, 0.7682, 0.3937, 1.0996]])
```

```
d_k = keys.shape[-1]
attn_weights_2 = torch.softmax(attn_scores_2 / d_k**0.5, dim=-1)
print("Scaled and Normalized Attention weights : ", attn_weights_2)
print("Sum of attention weights : ", attn_weights_2.sum())
print("Dimension of key vector : ", d_k)
```

```
Scaled and Normalized Attention weights :  tensor([0.1500, 0.2264,
0.2199, 0.1311, 0.0906, 0.1820])
Sum of attention weights :  tensor(1.)
Dimension of key vector :  2
```

WHY DIVIDE BY SQRT (DIMENSION)

```python
import torch

# Define the tensor
tensor = torch.tensor([0.1, -0.2, 0.3, -0.2, 0.5])

# Apply softmax without scaling
softmax_result = torch.softmax(tensor, dim=-1)
print("Softmax without scaling : ", softmax_result)

# Multiply the tensor by 8 and then apply softmax
scaled_tensor = tensor * 8
softmax_scaled_result = torch.softmax(scaled_tensor, dim=-1)
print("Softmax after scaling (tensor * 8) : ", softmax_scaled_result)
```

```
Softmax without scaling :  tensor([0.1925, 0.1426, 0.2351, 0.1426,
0.2872])
Softmax after scaling (tensor * 8) :  tensor([0.0326, 0.0030, 0.1615,
0.0030, 0.8000])
```

```python
import numpy as np


# Function to compute variance before and after scaling
def compute_variance(dim, num_trials=1000):
    dot_products = []
    scaled_dot_products = []

    # Generate multiple random vectors and compute dot products
    for _ in range(num_trials):
        q = np.random.randn(dim)
        k = np.random.randn(dim)

        # Compute dot product
        dot_product = np.dot(q, k)
        dot_products.append(dot_product)

        # Scale the dot product by sqrt(dim)
        scaled_dot_product = dot_product / np.sqrt(dim)
        scaled_dot_products.append(scaled_dot_product)

    # Calculate variance of the dot products
    variance_before_scaling = np.var(dot_products)
    variance_after_scaling = np.var(scaled_dot_products)
```

```
    return variance_before_scaling, variance_after_scaling


# For dimension 5
variance_before_5, variance_after_5 = compute_variance(5)
print(f"Variance before scaling (dim=5): {variance_before_5}")
print(f"Variance after scaling (dim=5): {variance_after_5}")

# For dimension 20
variance_before_100, variance_after_100 = compute_variance(100)
print(f"Variance before scaling (dim=100): {variance_before_100}")
print(f"Variance after scaling (dim=100): {variance_after_100}")
```

```
Variance before scaling (dim=5): 4.494116758875627
Variance after scaling (dim=5): 0.8988233517751253
Variance before scaling (dim=100): 98.4859830120943
Variance after scaling (dim=100): 0.984859830120943
```

```
context_vec_2 = attn_weights_2 @ values
print(context_vec_2)
```

```
tensor([0.3061, 0.8210])
```

IMPLEMENTING A COMPACT SELF ATTENTION PYTHON CLASS

```
import torch.nn as nn


class SelfAttention_v1(nn.Module):
    def __init__(self, d_in, d_out, qkv_bias=False):
        super().__init__()
        # self.W_query = nn.Parameter(torch.rand(d_in, d_out))
        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
        # self.W_key = nn.Parameter(torch.rand(d_in, d_out))
        self.W_key = nn.Linear(d_in, d_out, bias=qkv_bias)
        # self.W_value = nn.Parameter(torch.rand(d_in, d_out))
        self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)

    def forward(self, x):
        keys = self.W_key(x)
        queries = self.W_query(x)
        values = self.W_value(x)

        attn_scores = queries @ keys.T  # omega

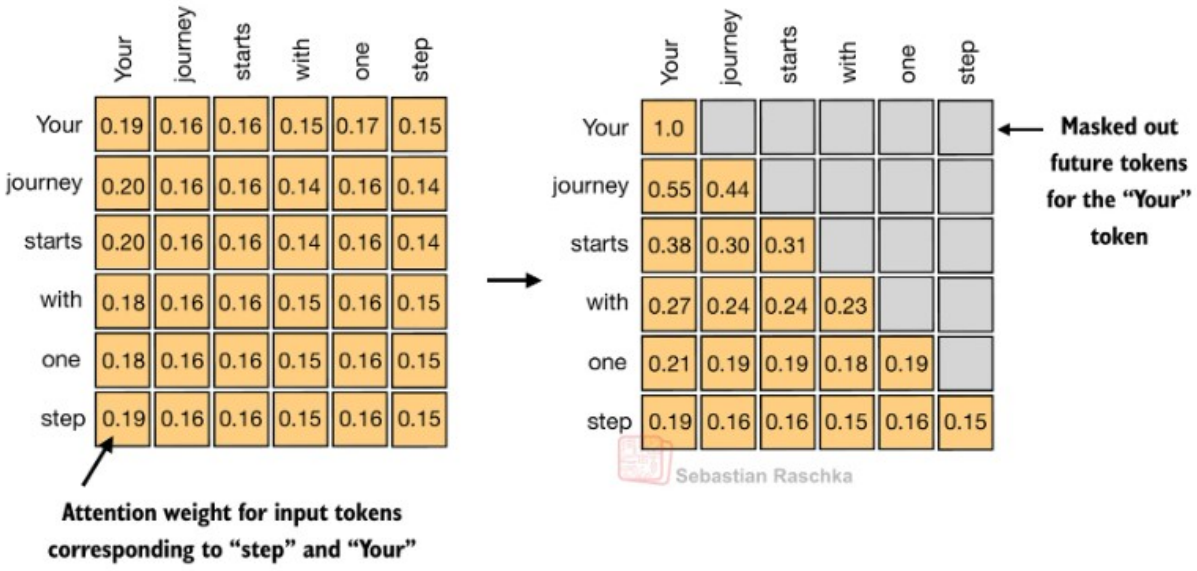
        # Scale the attention scores by the square root of the
dimension of the key vectors And Normalize the attention scores using
softmax
        attn_weights = torch.softmax(attn_scores / keys.shape[-1] **
0.5, dim=-1)
```

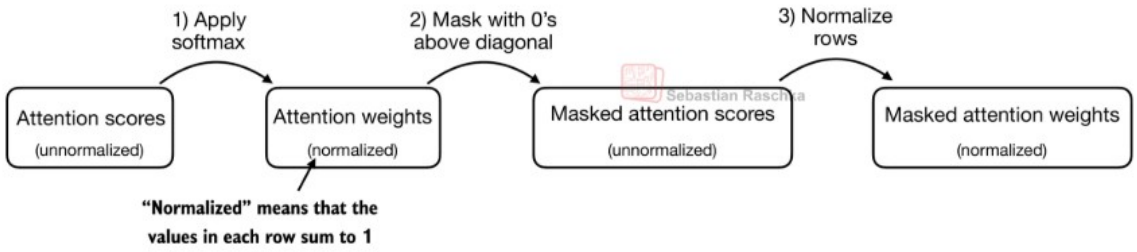
```
        context_vec = attn_weights @ values
        return context_vec

torch.manual_seed(123)
sa_v1 = SelfAttention_v1(d_in, d_out)
print(sa_v1(inputs))
```

```
tensor([[-0.5337, -0.1051],
        [-0.5323, -0.1080],
        [-0.5323, -0.1079],
        [-0.5297, -0.1076],
        [-0.5311, -0.1066],
        [-0.5299, -0.1081]], grad_fn=<MmBackward0>)
```

HIDING FUTURE WORDS WITH CAUSAL ATTENTION - MASKED SELF-ATTENTION






```
queries = sa_v1.W_query(inputs)
keys = sa_v1.W_key(inputs)
values = sa_v1.W_value(inputs)
```

```
attention_scores = queries @ keys.T
print("Attention Scores : ", attention_scores)

attention_weights = torch.softmax(attention_scores / keys.shape[-1] **
0.5, dim=-1)
print("Attention Weights : ", attention_weights)
```

```
Attention Scores :  tensor([[0.3111, 0.3479, 0.3471, 0.1714, 0.2350,
0.1928],
        [0.1655, 0.2602, 0.2576, 0.1445, 0.1384, 0.1790],
        [0.1667, 0.2602, 0.2577, 0.1443, 0.1391, 0.1784],
        [0.0510, 0.1080, 0.1064, 0.0643, 0.0476, 0.0835],
        [0.1415, 0.1875, 0.1863, 0.0987, 0.1121, 0.1174],
        [0.0476, 0.1192, 0.1171, 0.0731, 0.0477, 0.0966]],
       grad_fn=<MmBackward0>)
Attention Weights :  tensor([[0.1717, 0.1762, 0.1761, 0.1555, 0.1627,
0.1579],
        [0.1636, 0.1749, 0.1746, 0.1612, 0.1605, 0.1652],
        [0.1637, 0.1749, 0.1746, 0.1611, 0.1606, 0.1651],
        [0.1636, 0.1704, 0.1702, 0.1652, 0.1632, 0.1674],
        [0.1667, 0.1722, 0.1721, 0.1618, 0.1633, 0.1639],
        [0.1624, 0.1709, 0.1706, 0.1654, 0.1625, 0.1682]],
       grad_fn=<SoftmaxBackward0>)
```

```
context_len = attention_weights.shape[0]
print("Context Length : ", context_len)

simple_mask = torch.tril(torch.ones(context_len, context_len))
print("Simple Mask : ", simple_mask)

masked_attention_scores = attention_weights * simple_mask
print("Masked Attention Scores : ", masked_attention_scores)

normalized_masked_attention_scores  = masked_attention_scores /
masked_attention_scores.sum(dim=-1, keepdim=True)
print("Normalized Masked Attention Scores : ",
normalized_masked_attention_scores)
```

```
Context Length :  6
Simple Mask :  tensor([[1., 0., 0., 0., 0., 0.],
        [1., 1., 0., 0., 0., 0.],
        [1., 1., 1., 0., 0., 0.],
        [1., 1., 1., 1., 0., 0.],
        [1., 1., 1., 1., 1., 0.],
        [1., 1., 1., 1., 1., 1.]])
Masked Attention Scores :  tensor([[0.1717, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000],
        [0.1636, 0.1749, 0.0000, 0.0000, 0.0000, 0.0000],
        [0.1637, 0.1749, 0.1746, 0.0000, 0.0000, 0.0000],
```

```
        [0.1636, 0.1704, 0.1702, 0.1652, 0.0000, 0.0000],
        [0.1667, 0.1722, 0.1721, 0.1618, 0.1633, 0.0000],
        [0.1624, 0.1709, 0.1706, 0.1654, 0.1625, 0.1682]],
       grad_fn=<MulBackward0>)
Normalized Masked Attention Scores :  tensor([[1.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000],
        [0.4833, 0.5167, 0.0000, 0.0000, 0.0000, 0.0000],
        [0.3190, 0.3408, 0.3402, 0.0000, 0.0000, 0.0000],
        [0.2445, 0.2545, 0.2542, 0.2468, 0.0000, 0.0000],
        [0.1994, 0.2060, 0.2058, 0.1935, 0.1953, 0.0000],
        [0.1624, 0.1709, 0.1706, 0.1654, 0.1625, 0.1682]],
       grad_fn=<DivBackward0>)

print(attention_weights)

tensor([[0.1717, 0.1762, 0.1761, 0.1555, 0.1627, 0.1579],
        [0.1636, 0.1749, 0.1746, 0.1612, 0.1605, 0.1652],
        [0.1637, 0.1749, 0.1746, 0.1611, 0.1606, 0.1651],
        [0.1636, 0.1704, 0.1702, 0.1652, 0.1632, 0.1674],
        [0.1667, 0.1722, 0.1721, 0.1618, 0.1633, 0.1639],
        [0.1624, 0.1709, 0.1706, 0.1654, 0.1625, 0.1682]],
       grad_fn=<SoftmaxBackward0>)

# Create a upper Triangular matrix for making the values=1 for all the
elements above the diagonal and 0 for all the elements below the
diagonal (including the diagonal)

mask = torch.triu(torch.ones(context_len, context_len), diagonal=1)
print("Mask : \n", mask)

# Apply the mask to the attention scores and replace 1 with -inf
# mask.bool() will convert the mask to a boolean tensor where 1s
become True and 0s become False. Then, we use masked_fill to replace
the True values in attention_weights with -inf.

masked_attention_scores_new =
attention_weights.masked_fill(mask.bool(), -torch.inf)
print("Masked Attention Scores (with -inf) : \n",
masked_attention_scores_new)

# Now we apply softmax to the masked attention scores to get the final
attention weights. The -inf values will result in 0 probabilities
after softmax. as e to the power of -inf is 0

final_attention_weights = torch.softmax(masked_attention_scores_new /
keys.shape[-1] ** 0.5 , dim=-1)
print("Final Attention Weights (scaled and normalized via softmax): \
n", final_attention_weights)

Mask :
 tensor([[0., 1., 1., 1., 1., 1.],
```

```
        [0., 0., 1., 1., 1., 1.],
        [0., 0., 0., 1., 1., 1.],
        [0., 0., 0., 0., 1., 1.],
        [0., 0., 0., 0., 0., 1.],
        [0., 0., 0., 0., 0., 0.]])
Masked Attention Scores (with -inf) :
 tensor([[0.1717,   -inf,   -inf,   -inf,   -inf,   -inf],
        [0.1636, 0.1749,   -inf,   -inf,   -inf,   -inf],
        [0.1637, 0.1749, 0.1746,   -inf,   -inf,   -inf],
        [0.1636, 0.1704, 0.1702, 0.1652,   -inf,   -inf],
        [0.1667, 0.1722, 0.1721, 0.1618, 0.1633,   -inf],
        [0.1624, 0.1709, 0.1706, 0.1654, 0.1625, 0.1682]],
       grad_fn=<MaskedFillBackward0>)
Final Attention Weights (scaled and normalized via softmax):
 tensor([[1.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
        [0.4980, 0.5020, 0.0000, 0.0000, 0.0000, 0.0000],
        [0.3316, 0.3342, 0.3342, 0.0000, 0.0000, 0.0000],
        [0.2493, 0.2505, 0.2505, 0.2496, 0.0000, 0.0000],
        [0.1999, 0.2007, 0.2007, 0.1992, 0.1994, 0.0000],
        [0.1662, 0.1672, 0.1671, 0.1665, 0.1662, 0.1668]],
       grad_fn=<SoftmaxBackward0>)
```

```
# Example of Dropout layer in PyTorch

torch.manual_seed(123)
dropout = torch.nn.Dropout(0.5)  # A
example = torch.ones(6, 6)  # B
print(dropout(example))
```

```
tensor([[2., 2., 0., 2., 2., 0.],
        [0., 0., 0., 2., 0., 2.],
        [2., 2., 2., 2., 0., 2.],
        [0., 2., 2., 0., 0., 2.],
        [0., 2., 0., 2., 0., 2.],
        [0., 2., 2., 2., 2., 0.]])
```

```
print("Final Attention Weights (Masked , scaled and normalized via
softmax): \n", final_attention_weights)
print("Final Attention Weights after applying Dropout: \n",
dropout(final_attention_weights))
```

```
Final Attention Weights (Masked , scaled and normalized via softmax):
 tensor([[1.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
        [0.4980, 0.5020, 0.0000, 0.0000, 0.0000, 0.0000],
        [0.3316, 0.3342, 0.3342, 0.0000, 0.0000, 0.0000],
        [0.2493, 0.2505, 0.2505, 0.2496, 0.0000, 0.0000],
        [0.1999, 0.2007, 0.2007, 0.1992, 0.1994, 0.0000],
        [0.1662, 0.1672, 0.1671, 0.1665, 0.1662, 0.1668]],
       grad_fn=<SoftmaxBackward0>)
Final Attention Weights after applying Dropout:
```

```
 tensor([[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
        [0.0000, 1.0040, 0.0000, 0.0000, 0.0000, 0.0000],
        [0.6632, 0.6685, 0.6683, 0.0000, 0.0000, 0.0000],
        [0.4987, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
        [0.3999, 0.4014, 0.4014, 0.0000, 0.3989, 0.0000],
        [0.0000, 0.0000, 0.3343, 0.0000, 0.0000, 0.3337]],
       grad_fn=<MulBackward0>)
```

IMPLEMENTING A COMPACT CAUSAL ATTENTION CLASS

```
inputs = torch.tensor(
    [
        [0.43, 0.15, 0.89], # Your     (x^1)
        [0.55, 0.87, 0.66], # journey  (x^2)
        [0.57, 0.85, 0.64], # starts   (x^3)
        [0.22, 0.58, 0.33], # with     (x^4)
        [0.77, 0.25, 0.10], # one      (x^5)
        [0.05, 0.80, 0.55], # step     (x^6)
    ]
)

batch_inputs = torch.stack([inputs, inputs], dim=0)  # Create a batch
of size 2
print("Batch Input Shape:", batch_inputs.shape)  # Should be (2, 6, 3)

Batch Input Shape: torch.Size([2, 6, 3])

class CasualAttention(nn.Module):
    def __init__(self, d_in, d_out, context_len, dropout,
qkv_bias=False):
        super().__init__()
        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_key = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.dropout = nn.Dropout(dropout)
        self.register_buffer("mask",
torch.triu(torch.ones(context_len, context_len), diagonal=1))

    def forward(self, x):
        batch_size,num_tokens,d_in = x.shape
        print(f"Input vector Dimensions Info - Batch Size:
{batch_size}, Number of Tokens: {num_tokens}, Input Dimension:
{d_in}")

        keys = self.W_key(x)
        print("Keys Shape:", keys.shape)  # Should be (batch_size,
num_tokens, d_out)

        queries = self.W_query(x)
        print("Queries Shape:", queries.shape)  # Should be
```

```
(batch_size, num_tokens, d_out)

        values = self.W_value(x)
        print("Values Shape:", values.shape)  # Should be (batch_size,
num_tokens, d_out)

        attention_scores = queries @ keys.transpose(1,2)
        # (1,2) because we want to transpose the last two dimensions
of keys to perform the batch matrix multiplication correctly. This
ensures that for each batch, we compute the dot product between the
query and key vectors across all tokens.

        attention_scores.masked_fill_(
            self.mask.bool()[:num_tokens,:num_tokens], -torch.inf
            # :num_tokens,:num_tokens is used to ensure that the mask
is applied correctly for the current number of tokens in the input
sequence, especially when dealing with batches of varying sequence
lengths.

        )

        attention_weights = torch.softmax(attention_scores /
keys.shape[-1] ** 0.5 , dim=-1)

        # adding the droput layer
        attention_weights = self.dropout(attention_weights)

        context_vectors = attention_weights @ values

        return context_vectors

print("Input dimension of vectors : ",d_in)
print("Desired output dimension of Resulting Context vectors :
",d_out)
print("Batch input shape : 2 - no of Batches , 6 - no of I/p Tokens ,
3 - I/p dim : ",batch_inputs.shape)
```

```
Input dimension of vectors :  3
Desired output dimension of Resulting Context vectors :  2
Batch input shape : 2 - no of Batches , 6 - no of I/p Tokens ,  3 -
I/p dim :  torch.Size([2, 6, 3])
```

```
torch.manual_seed(123)
context_length = batch_inputs.shape[1]
print("Context Length : ", context_length)

# Creating a class of Casual Attention with the given parameters
casual_attention = CasualAttention(d_in=d_in, d_out=d_out,
context_len=context_length, dropout=0.0, qkv_bias=False)

# passing the batch input to the casual attention class and getting
```

```
the context vectors as output
context_vectors = casual_attention(x=batch_inputs)

# Printing the shape of the resulting context vectors
print("Context Vectors Shape : ", context_vectors.shape)  # Should be
(batch_size, num_tokens, d_out)
print("Context Vectors : \n ", context_vectors)
```

```
Context Length :  6
Input vector Dimensions Info - Batch Size: 2, Number of Tokens: 6,
Input Dimension: 3
Keys Shape: torch.Size([2, 6, 2])
Queries Shape: torch.Size([2, 6, 2])
Values Shape: torch.Size([2, 6, 2])
Context Vectors Shape :  torch.Size([2, 6, 2])
Context Vectors :
  tensor([[[-0.4519,  0.2216],
         [-0.5874,  0.0058],
         [-0.6300, -0.0632],
         [-0.5675, -0.0843],
         [-0.5526, -0.0981],
         [-0.5299, -0.1081]],

        [[-0.4519,  0.2216],
         [-0.5874,  0.0058],
         [-0.6300, -0.0632],
         [-0.5675, -0.0843],
         [-0.5526, -0.0981],
         [-0.5299, -0.1081]]], grad_fn=<UnsafeViewBackward0>)
```

```
# 3 input tokens having 6 dimensions each , d_in = 6
input = 3*6

# Dimension of query, key and value weught matrices
Wq , Wk, Wv = 6*6

# d_out = 6 - output dimension of the context vector
out_dim = 6

# Number of heads in the multi-head attention
no_of_heads = 2

# d_out = no_of_heads * d_out_per_head
# 6 // 2 = 3
d_out_per_head = out_dim // no_of_heads
d_out_per_head = 3

# so , in multi-head attention , we will have 2 heads , each head will
have 3 dimensions for the context vector
```

```
# so we transform the input vector of 6 dimensions into 2 heads of 3 dimensions each for the context vector
Q_K_V_input_shape = 2*3*6
# (batch_size ,  no_of_tokens ,  out_dim)
Q_K_V_new_input_shape = 2*3*2*3
# (batch_size ,  no_of_tokens ,  no_of_heads ,  d_out_per_head)

# Transpose: (b, num_tokens, no_of_heads, d_out_per_head) -> (b, no_of_heads, num_tokens, d_out_per_head)
initial_k = 2*3*2*3
k = initial_k.transpose(1, 2)

#  so , the shape of the keys after transposing will be (batch_size, no_of_heads, num_tokens, d_out_per_head)
#  we do this because we want to perform the dot product between the query and key vectors for each head separately (parallel computation) so we group on basis on no of heads, and then combine the results to get the final context vector.
k = 2*2*3*3

# Similarly, we will do the same for the query and value vectors as well.
q = 2*2*3*3
v = 2*2*3*3
# 2 - batch size , 2 - no of heads , 3 - no of tokens , 3 - d_out_per_head

attn_Scores = q * k.transpose(2, 3)
# q = (2*2*3*3 , 2 = batch size , 2 - no of heads , 3 - no of tokens , 3 - d_out_per_head)
# k.transpose(2, 3) = (2*2*3*3 , 2 - batch size , 2 - no of heads , 3 - d_out_per_head , 3 - no of tokens)

res_attn_Scores = 2*2*3*3
# (2 - batch size , 2 - no of heads , 3 - no of tokens , 3 - no of tokens)
```